

GPU Modular Arithmetic Unit with Barrett Reduction

Akhil Teja Chirra
Dept. of Electrical and
Computer Engineering
UC San Diego
achirra@ucsd.edu

Vagda Sameera Talari
Dept. of Electrical and
Computer Engineering
UC San Diego
vtalari@ucsd.edu

Abhijit Gupta
Dept. of Electrical and
Computer Engineering
UC San Diego
akg008@ucsd.edu

Abstract—We implement a prime-agnostic modular arithmetic engine over large prime fields entirely from scratch in pure CUDA, targeting an NVIDIA Tesla T4 GPU (sm_75). The engine supports modular multiplication, modular exponentiation, and Barrett reduction, applied to a Number Theoretic Transform (NTT)—the modular-arithmetic core of lattice-based post-quantum cryptography (Kyber, Dilithium) and zero-knowledge proof systems. We benchmark GPU implementations against single-threaded CPU equivalents using the identical Barrett C++ code path. Measured results on a Colab T4: Barrett modmul achieves a $569\times$ GPU speedup (131 Gops/s), the NTT kernel peaks at $52\times$ speedup at $n = 2^{18}$, and batch modular exponentiation over the 64-bit Goldilocks prime achieves $277\times$ speedup with zero correctness failures. We analyze the performance-security trade-offs including timing side-channel exposure and the compute-to-memory-bandwidth transition at scale. Code is available at <https://github.com/achirra-cipher/ece268-gpu-modarith>.

Index Terms—Barrett reduction, modular arithmetic, GPU cryptography, CUDA, Number Theoretic Transform, post-quantum cryptography, NTT, modular exponentiation

I. INTRODUCTION

Modern cryptographic workloads—lattice-based post-quantum schemes (Kyber [1], Dilithium [2]) and zero-knowledge proof systems (Plonky2 [3])—are dominated by modular arithmetic over large prime fields. The Number Theoretic Transform (NTT), the algebraic analogue of the FFT over a finite field, accounts for the majority of compute in these schemes. An NTT of size n consists of $\log_2 n$ stages, each with $n/2$ independent butterfly operations; the workload is massively data-parallel and each butterfly requires two modular multiplications, making NTT throughput directly proportional to modmul throughput.

GPUs expose thousands of parallel cores and are natural accelerators for such workloads. Prior work has demonstrated GPU acceleration of elliptic-curve scalar multiplication, hash functions, and symmetric ciphers, but modular arithmetic engines targeting the prime fields used in post-quantum lattice schemes and ZK-proof systems remain less explored in the pure-CUDA setting. A key challenge is modular reduction: standard integer division is slow on GPUs—the hardware lacks a native 64-bit integer divide instruction—and data-dependent division latency introduces a potential timing side-channel in cryptographic contexts. Barrett reduction [4] addresses both

problems by replacing division with a multiply-shift sequence using a precomputed constant μ , making the reduction branch-light, divide-free, and GPU-efficient.

This paper presents a pure-CUDA modular arithmetic engine that implements: (i) Barrett reduction for two prime fields, (ii) modular multiplication (modmul), (iii) modular exponentiation (modexp) via square-and-multiply, and (iv) modular inverse (modinv) via Fermat’s little theorem. The engine is exercised through a GPU NTT and a batch modexp demo over the 64-bit Goldilocks prime. We benchmark GPU versus single-threaded CPU performance and analyze correctness, throughput, and security trade-offs. The entire implementation is written in pure CUDA/C++ with no PyCUDA, CuPy, or other Python GPU wrappers, qualifying for the bonus pure-CUDA track.

II. BACKGROUND

A. Barrett Reduction

Given an integer $x \in [0, p^2)$ and a modulus p , Barrett reduction computes $x \bmod p$ without division. A constant $\mu = \lfloor 2^k/p \rfloor$ is precomputed offline. The reduction then proceeds as:

$$q = (x \cdot \mu) \gg k \quad (1)$$

$$r = x - q \cdot p \quad (2)$$

$$r -= p \quad \text{if } r \geq p \quad (3)$$

Step (1) estimates $\lfloor x/p \rfloor$, which is off by at most 1. Step (2) produces $r \in [0, 2p)$; the single conditional subtraction in (3) brings r into $[0, p)$. This reduces modular reduction to two multiplications and one shift—no hardware division required.

The constant μ is shared across all threads and resides in constant cache or a register broadcast. On sm_75 (T4), the 64-bit high-half multiply maps to the `__umul64hi` hardware intrinsic—a single instruction—making the hot path two 64-bit multiplies, cheap and warp-uniform with no thread divergence from the reduction itself.

B. Number Theoretic Transform

The NTT over $\mathbb{F}_p$ requires $p \equiv 1 \pmod{n}$ so that n -th roots of unity exist in the field [6]. The Cooley-Tukey butterfly at

TABLE I
PRIME FIELDS USED

Name	Prime	Bits	Role
P32	$119 \cdot 2^{23} + 1$	30	Primary (NTT)
P64	$2^{64} - 2^{32} + 1$	64	Bonus (modexp, zk)

each stage computes:

$$A[u] = (u + \omega \cdot v) \bmod p \quad (4)$$

$$A[v] = (u - \omega \cdot v) \bmod p \quad (5)$$

where ω is a twiddle factor. Each butterfly requires two modular multiplications ((4)–(5)), so NTT throughput is directly proportional to modmul throughput. The $\log_2 n$ stages each consist of $n/2$ independent butterfly pairs, making the NTT ideal for GPU parallelism.

C. Barrett vs. Montgomery

Montgomery multiplication [5] is an alternative that amortizes the division cost over multiple multiplications by working in a Montgomery domain. It typically requires one fewer conditional subtraction per reduction in tight modexp loops, because the correction term is naturally absorbed into the multiply-and-add chain. However, Montgomery requires explicit conversion of operands into Montgomery form ($a \rightarrow aR \bmod p$, where $R = 2^{64}$) and back out afterward. For standalone modular exponentiation this conversion cost amortizes over many multiplications on the same value.

For NTT, Barrett is the better choice: the n input elements must stay in natural residue form across all $\log_2 n$ butterfly stages. Switching to Montgomery would require two additional $O(n)$ passes—one to convert the entire array into Montgomery domain before stage 1, and one to convert back after the final stage. As we show in Section VI, the NTT is already memory-bandwidth-bound at large n ; these two extra passes would directly reduce effective speedup. Our engine is prime-agnostic: any prime p requires only recomputing $\mu = \lfloor 2^k/p \rfloor$ with no other structural changes.

III. IMPLEMENTATION

A. Prime Field Selection

We use two prime fields (Table I). **P32** ($119 \cdot 2^{23} + 1$) is the primary NTT benchmark prime: 30-bit, NTT-friendly (2-adicity 23, generator $g = 3$), with all products fitting in 64 bits, enabling Barrett with native `__uint128_t`. **P64** ($2^{64} - 2^{32} + 1$, the Goldilocks prime) is used in real zero-knowledge proof systems (Plonky2) and requires 128-bit intermediates on device—the bonus large-prime path.

B. Core Arithmetic: `modarith.cuh`

All arithmetic lives in a single CUDA header `modarith.cuh` marked `__host__ __device__`, enabling the same Barrett and modmul logic to compile and run both on the CPU (for local correctness testing) and on the GPU (for production). The key device-side optimization

uses the `__umul64hi` intrinsic—a single `sm_75` instruction that computes the high 64 bits of a 128-bit product:

```
// Pseudocode illustrates algorithm structure
// Actual implementation: cuda/include/modarith.cuh

__host__ __device__ static inline u64
barrett32_reduce(u64 x, const Barrett32& B) {
    // q = floor(x / p), off by at most 1
    u64 q = ((__uint128_t)x * B.mu) >> B.k;
    u64 r = x - q * B.p; // r in [0, 2p)
    if (r >= B.p) r -= B.p; // single correction
    return r;
}

__device__ static inline u64
mulmod32(u64 a, u64 b, const Barrett32& B) {
    u64 hi = __umul64hi(a, b); // 1 sm_75 instruction
    u64 lo = a * b;
    return barrett32_reduce((hi << 32) | (lo >> 32), B);
}
```

Modular exponentiation is implemented via square-and-multiply, iterating over the bits of the exponent (up to 64 bits for 64-bit exponents; 30 bits for P32 with $p - 2$ as exponent). Modular inverse uses Fermat’s little theorem: $\text{inv}(a) = a^{p-2} \bmod p$, implemented by calling `modexp` with exponent $p - 2$. Both operations are fully portable across P32 and P64 via the same Barrett interface.

C. GPU NTT Kernel

The NTT is implemented as an iterative Cooley-Tukey transform, with one CUDA thread handling one butterfly pair per stage:

```
// Pseudocode actual kernel: cuda/include/ntt_gpu.cuh
// Driver in cuda/src/ntt.cu

__global__ void k_ntt_stage(
    u64* A, int half, const u64* roots,
    int n, const Barrett32 B) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid >= n/2) return;
    int j = tid % half;
    int u_i = (tid/half)*2*half + j;
    int v_i = u_i + half;
    u64 tw = roots[(n / (2*half)) * j]; // precomputed
    u64 u = A[u_i];
    u64 v = mulmod32(A[v_i], tw, B);
    A[u_i] = modadd(u, v, B.p);
    A[v_i] = modsub(u, v, B.p);
}
```

The kernel is launched $\log_2 n$ times—once per NTT stage—with $\lceil n/2/256 \rceil$ blocks of 256 threads each (block size chosen to maximise occupancy on `sm_75` with its 2048 resident threads per SM). Each launch implicitly synchronises the entire device, providing stage-to-stage coherence without explicit barriers.

Bit-reversal permutation. Before the forward NTT, a separate kernel rearranges the input array into bit-reversed order.

Each thread i swaps $A[i]$ with $A[\text{bitrev}(i)]$ when $\text{bitrev}(i) > i$, computed with a standard $\log_2 n$ -step bit-flip loop. The permutation runs in a single kernel launch and is counted in the “GPU total” timing column of Table III.

Inverse NTT (INTT). The INTT reuses the same `k_ntt_stage` kernel with inverse twiddle factors $\omega^{-1} = \text{modpow}(\omega, p-1-n/p, B)$ and a final element-wise scaling by $n^{-1} \bmod p$ (one modmul per element). This design avoids code duplication: the forward and inverse transforms share the identical butterfly implementation. Correctness is verified by the round-trip identity $\text{INTT}(\text{NTT}(x)) = x$ for all tested sizes up to $n = 2^{14}$ (Section IV).

Choosing Barrett over Montgomery avoids domain-conversion passes between stages, which is critical because NTT becomes memory-bandwidth-bound at large n .

D. Repository Structure

The complete codebase is organized as follows:

- `cuda/include/modarith.cuh` — Barrett, modmul, modexp, modinv (host+device)
- `cuda/include/params.h` — Prime parameters and precomputed μ
- `cuda/src/ntt.cu` — GPU NTT butterfly kernels + bit-reversal
- `cuda/src/bench.cu` — CUDA-event benchmark harness
- `cuda/src/modexp_demo.cu` — Batch modexp over P64
- `python/oracle.py` — Ground-truth test vectors via sympy
- `colab/run_on_colab.ipynb` — End-to-end Colab T4 notebook

E. Development Workflow

Development was conducted on an Apple Silicon Mac (arm64, no CUDA). The `__host__ __device__` dual-compilation strategy allows the identical Barrett and modmul code to be compiled as plain C++ with the host compiler (clang on Mac, g++ on Linux/Colab) and unit-tested locally against Python/sympy ground truth, with no GPU required. Only after the CPU unit tests pass does the code move to GPU compilation.

The workflow proceeds in three phases. *Phase 1 (local)*: `host_test.cpp` includes `modarith.cuh` directly, runs 2M+ random test cases against 128-bit ground truth, and compares against `oracle.py` output. *Phase 2 (Colab compile)*: the `.cu` sources are compiled with `nvcc -arch=sm_75 -O3` on a Colab T4 instance; if Phase 1 passed, compilation errors almost always indicate a GPU-specific intrinsic issue rather than an arithmetic bug. *Phase 3 (GPU verify)*: the compiled binary runs the same test suite on GPU, and outputs are compared byte-for-byte with the Phase 1 CPU reference. This three-phase approach reduced GPU debugging time significantly, as the expensive Colab session was only needed for performance measurement, not arithmetic validation.

TABLE II
BARRETT MODMUL THROUGHPUT ($n = 4\text{M}$, P32)

Metric	CPU (1 thread)	GPU kernel (T4)	Speedup
Throughput	0.23 Gops/s	131.4 Gops/s	569×
Kernel wall time	1161 ms	2.04 ms	569×

IV. CORRECTNESS & VALIDATION

We enforce a three-layer validation protocol: GPU C++ (nvcc) $\equiv$ CPU C++ (clang) $\equiv$ Python (sympy). The protocol is designed so that no two of the three layers share implementation code or arithmetic libraries: the GPU and CPU paths share `modarith.cuh` but differ in compiler and hardware; the Python layer uses sympy’s arbitrary-precision integer arithmetic, which is completely independent. Agreement across all three layers therefore provides strong evidence of correctness.

Barrett32 / Barrett64. We generate 2 million+ random operand pairs (a, b) uniformly distributed in $[0, p)$ and verify that $\text{modmul}(a, b)$ matches $a \cdot b \bmod p$ computed via `__uint128_t` ground truth. Boundary operands ($a = 0$, $a = p - 1$, products near p^2) are tested separately. Both Barrett32 (P32) and Barrett64 (P64) pass with zero failures.

modexp / modinv. We test 100K random exponents, verifying $\text{modexp}(a, e, p) = a^e \bmod p$ against an independent square-and-multiply implemented with native 128-bit arithmetic. $\text{modinv}(a, p)$ is verified via the identity $a \cdot \text{inv}(a) \equiv 1 \pmod{p}$.

NTT correctness. The CPU NTT is validated by the round-trip identity $\text{INTT}(\text{NTT}(x)) = x$ for n up to 2^{14} , and by verifying that NTT-based cyclic convolution matches the naive $O(n^2)$ pointwise product for n up to 2^9 . The GPU forward NTT is then verified to match the CPU output element-wise for all sizes 2^4 through 2^{20} —every element of every transform matches exactly.

Batch modexp (GPU, P64). We run 1,048,576 independent 64-bit exponentiations over the Goldilocks prime on the T4 and compare element-wise against the CPU reference. Zero mismatches across the full 1M-element batch.

V. PERFORMANCE RESULTS

A. Experimental Setup

All benchmarks run on a Colab NVIDIA Tesla T4 (sm_75, 16 GB GDDR6, 2560 CUDA cores) versus a single CPU thread (Colab’s Xeon, ≈ 2 GHz effective clock). GPU timing uses `cudaEventRecord / cudaEventElapsedTime` and excludes Python/host overhead. The CPU uses the same Barrett C++ code compiled with the host C++ compiler (clang on Mac, g++ on Colab).

B. Barrett Modmul Throughput

Table II shows throughput for 4 million independent modular multiplications with 64 iterations per lane, fully saturating all T4 CUDA cores.

TABLE III
NTT KERNEL SPEEDUP (FORWARD NTT, P32)

n	GPU kernel	GPU total	CPU	Speedup
2^{10}	0.051 ms	0.100 ms	0.046 ms	$0.9\times$
2^{12}	0.061 ms	0.096 ms	0.193 ms	$3.2\times$
2^{14}	0.079 ms	0.163 ms	0.866 ms	$11\times$
2^{16}	0.141 ms	0.423 ms	3.82 ms	$27\times$
2^{18}	0.367 ms	1.312 ms	19.0 ms	$52\times$
2^{20}	2.54 ms	6.37 ms	90.1 ms	$35\times$
2^{22}	12.7 ms	27.3 ms	507 ms	$40\times$

TABLE IV
BATCH MODEXP OVER P64 GOLDSLOCKS ($n = 2^{20}$)

CPU	GPU (T4)	Speedup	Mismatches
955.8 ms	3.4 ms	$277\times$	0

The $569\times$ speedup reflects the embarrassingly parallel nature of independent modmul: one thread per element, no inter-thread communication, and all 2560 T4 CUDA cores kept fully saturated.

C. NTT Latency vs. Array Size

Table III shows NTT kernel latency and speedup across array sizes.

Three important observations emerge. (i) At $n = 2^{10}$, the GPU kernel is *slower* than a single CPU core ($0.9\times$). The primary cause is kernel-launch latency: CUDA’s `cudaEventRecord/cudaEventElapsedTime` measurements reveal a fixed per-launch overhead of roughly 40–60 μ s on the Colab T4, which dominates when useful work is only $n/2 = 512$ butterfly operations. The GPU only overtakes the CPU around $n = 2^{11}$ – 2^{12} , where $\log_2 n \approx 11$ stage launches together consume enough compute to amortize that fixed cost. (ii) Peak speedup ($52\times$, kernel-only) occurs at $n = 2^{18}$ (256K elements), the sweet spot where the working set fits in T4’s L2 cache and all 20 SMs are saturated. (iii) Beyond $n = 2^{18}$, speedup drops to ≈ 35 – $40\times$ as the working set overflows L2 and the kernel becomes memory-bandwidth-bound.

D. Batch Modular Exponentiation (P64)

Table IV shows results for 1M independent 64-bit modular exponentiations over the Goldilocks prime.

The $277\times$ speedup is compute-bound: each thread independently runs 64 square-and-multiply iterations with no inter-thread communication and near-uniform control flow—the archetypal GPU cryptographic workload.

VI. ANALYSIS

A. Why Modmul Speedup ($569\times$) $\gg$ NTT Speedup ($52\times$)

The modmul benchmark is perfectly embarrassingly parallel—zero dependencies between threads, perfectly coalesced memory access. The NTT butterfly pattern introduces two sources of inefficiency: (i) *inter-stage dependencies*: each of the $\log_2 n$ stages requires a separate kernel launch and

a full pass over device memory; (ii) *strided, non-coalesced global memory reads* in early stages, where butterfly pairs are separated by large strides. At large n , the NTT transitions from compute-bound to memory-bandwidth-bound, capping speedup well below the modmul case. The T4’s peak global memory bandwidth is approximately 320 GB/s; at $n = 2^{22}$ each stage reads and writes $n \times 8$ bytes = 32 MB, for a total of 22×64 MB = 1.4 GB of device memory traffic per transform, leaving little room for further compute scaling.

B. Roofline Analysis and Occupancy

To understand the compute vs. memory-bandwidth regime for each kernel, we apply a simplified Roofline model to the T4. The T4 delivers a peak FP32 throughput of approximately 8.1 TFLOPS and a peak global memory bandwidth of 320 GB/s, giving an arithmetic intensity ridge point of ≈ 25 FLOP/byte. Our kernels operate on 64-bit integers rather than floating-point, but the model is instructive as an order-of-magnitude guide.

The modmul kernel has near-perfect arithmetic intensity: each thread reads two 8-byte operands and performs ~ 5 integer operations (two 64-bit multiplies, a shift, and a subtract), yielding an intensity of roughly $5/16 \approx 0.3$ ops/byte—well below the ridge point. However, because all $n = 4$ M threads are launched simultaneously and there is no global memory traffic beyond the initial load of operands (each operand is used exactly once), the kernel saturates compute rather than memory bandwidth: it is effectively compute-bound at the warp level, with 2560 CUDA cores each handling independent modmul chains with no shared-memory or inter-warp communication.

The NTT kernel regime shifts with n . At small n ($\leq 2^{14}$), the working set ($n \times 8$ bytes ≤ 128 KB) fits in L1/shared memory across the 20 SMs, and the kernel is compute-bound. At $n = 2^{18}$ (2 MB), the working set fits in T4’s 4 MB L2 cache, maximising data reuse across the $\log_2 n = 18$ stage passes—this is where the $52\times$ peak speedup is achieved. Beyond $n = 2^{20}$ (8 MB), the working set overflows L2 and each stage requires a full round-trip to global DRAM, making the kernel strongly memory-bandwidth-bound. This explains why speedup drops from $52\times$ to 35 – $40\times$ at $n \geq 2^{20}$ despite the additional parallelism.

Occupancy on `sm_75` is also a limiting factor at small n . With 256 threads per block and $n/2$ total threads, the T4’s 20 SMs are not fully loaded below $n = 2^{13}$ (where the total thread count equals 4096 total threads $\ll 20 \times 2048 = 40960$ maximum resident threads). This partially explains the sub- $1\times$ speedup at $n = 2^{10}$ independently of launch overhead.

C. Barrett vs. Montgomery for NTT

Montgomery multiplication can reduce the per-reduction operation count in tight modexp loops, but for NTT, Barrett is the better choice: data stays in the natural residue domain across all butterfly stages. Montgomery would require converting the entire n -element array into Montgomery form before the NTT and back out afterward—two additional $O(n)$

TABLE V
SECURITY PROPERTIES OF THE IMPLEMENTATION

Property	Current status	Production fix
modexp timing	Data-dependent (branching on exponent bits)	Montgomery ladder / always-square
Barrett correction	Data-dependent (if $r \geq p$)	Branch-free $r -= p \ \& \ -(r \geq p)$
Field size	30-bit (P32), 64-bit (P64)	256-bit ECC or Kyber modulus ($q = 3329$)
Batch parallelism	High throughput for both parties	Secret keys must not be batched in non-constant-time kernels

passes over device memory. With large n where the NTT is already memory-bandwidth-bound, these extra passes would materially reduce effective speedup. Our engine is prime-agnostic: any p requires only recomputing $\mu = \lfloor 2^k/p \rfloor$.

D. Performance vs. Security Trade-offs

Our implementation is optimized for throughput benchmarking and carries two key security limitations.

Timing side channel in modexp. The square-and-multiply implementation branches on exponent bits (if $(e \ \& \ 1)$). If the exponent is secret (e.g., an RSA private key), this creates a data-dependent timing channel. A production-safe deployment would use a Montgomery ladder or always-square-and-multiply pattern with a constant exponent.

Data-dependent Barrett correction. The conditional subtraction if $(r \geq p) \ r -= p$ leaks one bit of the residual. The branch-free equivalent $r -= p \ \& \ -(u64)(r \geq p)$ eliminates this at negligible throughput cost.

Beyond these two points, the GPU batch-parallelism model itself has security relevance: the $277\times$ speedup for batch modexp (1M independent operations) reflects an ideal attacker-side advantage for brute-force or parallel exhaustive search. A single large modular exponentiation (e.g., one RSA-2048 decryption) remains inherently sequential; GPU advantage requires batching. Table V summarises the key security properties and the recommended fixes for a production deployment.

The field-size vs. throughput trade-off is measurable: moving from P32 (30-bit) to P64 (64-bit) reduces modmul-dominated speedup from $569\times$ to $277\times$; RSA-2048 or 256-bit ECC would require multi-limb arithmetic and further reduce raw throughput in exchange for security margin.

VII. CONCLUSION

We built a prime-agnostic GPU Barrett reduction engine in pure CUDA, validated by a three-layer correctness protocol ($\text{GPU} \equiv \text{CPU C++} \equiv \text{Python/sympy}$). Applied to the NTT, Barrett reduction achieves $52\times$ peak speedup at the compute-memory balance point ($n = 2^{18}$), while embarrassingly parallel modmul reaches $569\times$ and batch Goldilocks modexp reaches $277\times$. All results are obtained with zero correctness failures across more than one million verified output elements.

Three engineering insights emerged from this project. *First*, the `__host__ __device__` dual-compilation strategy—validating 2M+ arithmetic cases locally on a CPU before touching a GPU—is the single most effective tool for reducing debugging cost in GPU cryptography. *Second*, Barrett reduction is strictly preferable to Montgomery for NTT workloads, because it avoids two $O(n)$ domain-conversion memory passes between butterfly stages; at large n where the NTT is memory-bandwidth-bound, those passes would materially reduce effective speedup. *Third*, GPU speedup is not a fixed property of a workload—it is a function of problem size, memory access pattern, and kernel launch overhead. Our results show the GPU is actually *slower* than a single CPU core at $n = 2^{10}$, underscoring that GPU acceleration requires careful problem sizing.

The implementation is available in full at <https://github.com/achirra-cipher/ece268-gpu-modarith>, including all source files, the Colab benchmark notebook, and the Python oracle.

Future work includes: a negacyclic NTT variant for a direct Kyber/Dilithium polynomial multiplication tie-in; a shared-memory tiled NTT kernel to eliminate the per-stage global memory round-trip for small n ; an OpenMP multi-threaded CPU baseline for a fairer throughput comparison; and branch-free, constant-time Barrett correction ($r -= p \ \& \ -(u64)(r \geq p)$) for security-critical deployment where timing side-channels are a concern.

ACKNOWLEDGMENT

The authors thank the ECE 268 course staff at UC San Diego for project guidance and Google Colab for providing T4 GPU access used in all benchmarking.

REFERENCES

- [1] R. Avanzi et al., “CRYSTALS-Kyber (version 3.02),” NIST Post-Quantum Cryptography Finalist Specification, 2021. <https://pq-crystals.org/kyber/>
- [2] L. Ducas et al., “CRYSTALS-Dilithium (version 3.1),” NIST Post-Quantum Cryptography Finalist Specification, 2021. <https://pq-crystals.org/dilithium/>
- [3] Polygon Zero Team, “Plonky2: Fast Recursive Arguments with PLONK and FRI,” 2022. <https://github.com/mir-protocol/plonky2>
- [4] P. D. Barrett, “Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor,” in *Proc. CRYPTO*, Lecture Notes in Computer Science, vol. 263, pp. 311–323, Springer, 1987.
- [5] P. L. Montgomery, “Modular multiplication without trial division,” *Mathematics of Computation*, vol. 44, no. 170, pp. 519–521, 1985.
- [6] T. Pöppelmann and T. Güneysu, “Towards practical lattice-based public-key encryption on reconfigurable hardware,” in *Proc. SAC*, Lecture Notes in Computer Science, vol. 8282, pp. 68–85, Springer, 2013.